JMeter:

Plan de Pruebas “no funcionales”

MÓDULO: Taller de Testing y calidad de software

SEMANA: 7

Docente: Cynthia Torres Morales

Estudiante: Oliver Rubio Rauld

Índice

Introducción **3**

Desarrollo ***4***

- ***Información del sistema bajo prueba*** ***4***
- ***PRUEBA NO FUNCIONAL 1*** *(Carga)*
  - *Caso 1.1 Carga de página principal* ***5***
  - *Caso 1.2: Envío de Formularios Concurrentes* ***6***
  - *Caso 1.3: Búsquedas Concurrentes* ***7***
- ***PRUEBA NO FUNCIONAL 2*** *(Rendimiento)* ***9***
  - *Caso 2.1: Tiempo de Carga Inicial*
  - *Caso 2.2: Rendimiento de Validación RUT* ***11***
  - *Caso 2.3: Operaciones LocalStorage* ***12***
- ***PRUEBA NO FUNCIONAL 3*** *(Estrés)* ***14***
  - *Caso 3.1: Sobrecarga de Usuarios*
  - *Caso 3.2: Memoria y Recursos del Navegador* ***16***
  - *Caso 3.3: Concurrencia Extrema de Operaciones* ***17***
- ***CONFIGURACIÓN TÉCNICA JMETER*** ***20***
  - *Configuración Plan de Pruebas*
  - *Thread Groups*
  - *Samplers*
  - *Receptores Configurados* ***21***
- ***CAPTURAS***

Reflexiones finales ***25***

Conclusión ***27***

Bibliografía ***28***

Introducción

El presente informe documenta el diseño, planificación y ejecución de un conjunto de pruebas no funcionales orientadas a evaluar la calidad de una aplicación web de gestión de fichas médicas desarrollada con tecnologías HTML, CSS y JavaScript, cuyo almacenamiento de datos se realiza mediante localStorage. A diferencia de las pruebas funcionales, que verifican el cumplimiento de los requisitos operativos del sistema, las pruebas no funcionales se centran en atributos de calidad como el rendimiento, la estabilidad bajo carga y la resiliencia frente al estrés, factores críticos en el aseguramiento de la confiabilidad del software.

El proceso de validación descrito en este informe se estructura en tres grandes categorías de pruebas: pruebas de carga, orientadas a simular escenarios de uso normal con múltiples usuarios concurrentes; pruebas de rendimiento, enfocadas en medir la capacidad de respuesta del sistema en operaciones clave, tales como la validación del RUT chileno y las operaciones de lectura/escritura en localStorage; y pruebas de estrés, destinadas a identificar el punto de ruptura del sistema al someterlo a condiciones extremas de concurrencia y consumo de recursos. Cada categoría incluye tres casos de prueba específicos, cuidadosamente diseñados para reproducir condiciones realistas y recolectar métricas objetivas mediante la herramienta Apache JMeter, estándar de la industria para este tipo de evaluaciones.

El trabajo desarrollado no solo permitió generar un plan de pruebas integral, sino también aplicar un enfoque sistemático basado en los principios de software testing y en metodologías reconocidas para garantizar la calidad. Pese a las limitaciones técnicas encontradas en la ejecución práctica —derivadas de incompatibilidades con el entorno de pruebas—, el proceso documentado constituye una base sólida que, al ser replicada en un entorno configurado adecuadamente, permitirá obtener resultados concluyentes respecto a la robustez, eficiencia y escalabilidad del sistema. En este sentido, el informe representa un aporte significativo a la comprensión y aplicación de pruebas no funcionales en sistemas web críticos, aportando tanto valor académico como práctico para la formación en aseguramiento de calidad de software.

Desarrollo

Información del Sistema Bajo Prueba

- Sistema: Formulario de Ficha Médica
- País: Chile
- Tipo: Aplicación Web HTML/CSS/JavaScript
- Almacenamiento: localStorage (Cliente)
- Funcionalidades Principales: Registro y edición de pacientes, Validación de RUT chileno, Búsqueda por apellidos, Gestión de datos en localStorage.

PRUEBA NO FUNCIONAL 1: PRUEBA DE CARGA

Consideraciones: Esta prueba evalúa el comportamiento del sistema bajo condiciones normales de carga operativa, simulando múltiples usuarios concurrentes que acceden al formulario de registro de pacientes. Se busca verificar que el sistema mantenga tiempos de respuesta aceptable y no presente errores cuando varios usuarios interactúan simultáneamente con la aplicación.

Función probada (back end)	Simulación de operaciones LocalStorage
Objeto probado (front end)	Formulario de registro, Validación RUT, Búsqueda de pacientes
Responsable de la prueba	Oliver Rubio Rauld

Casos de Prueba de Carga

Caso 1.1: Carga de Página Principal

Descripción: Simular 50 usuarios concurrentes cargando la página principal

Configuración JMeter: Thread Group: 50 usuarios; Ramp-up: 10 segundos; Duración: 60 segundos.

Resultado Esperado: Tiempo de respuesta promedio < 2 segundos; 0% de errores; Throughput > 45 req/seg.

CICLOS (caso back end)

Ciclo	Ejecución de la prueba	Objetivo de la prueba	Resultados esperados	Resultados obtenidos
1.1.1	Carga inicial cache	Medir tiempo de primera carga de la página con 50 usuarios concurrentes	Tiempo respuesta <2s, 0% errores, Throughput>45 req/s	1759203758980,5,Medir_Carga_Completa,200,OK,2.1 Tiempo_Carga_Inicial 4-1,text,true,,36073,406,1,1,http://localhost:8080/formulario.html,,5,UTF-8,1,0,MacbookOliver,0,3
1.1.2	Carga con recursos embebidos	Verificar carga correcta de CSS y JavaScript inline	Todos los recursos cargados exitosamente sin errores 404	
1.1.3	Renderizado DOM	Validar que el DOM esté completamente construido	DOM Content Loaded < 1.5s, elementos críticos presentes	

JUEGO DE DATOS (caso back end)

Clave	Descripción
URL_BASE	http://localhost:8080/formulario.html
METODO_HTTP	GET
HEADER_ACCEPT	text/html, application/xhtml+xml, application/xml

HEADER_USER_AGENT	Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120.0.0.0
TIMEOUT_CONEXION	30000 ms
FOLLOW_REDIRECTS	true
RETRIEVE_RESOURCES	true (HTML, CSS, JS embebidos)

Caso 1.2: Envío de Formularios Concurrentes

Descripción: 30 usuarios enviando formularios simultáneamente

Configuración JMeter: Thread Group: 30 usuarios; Ramp-up: 5 segundos; Loop Count: 5

Resultado Esperado: Sin errores de JavaScript; Procesamiento exitoso del localStorage; Tiempo de respuesta < 1.5 segundos

CICLOS (caso back end)

Ciclo	**Ejecución de la prueba**	**Objetivo del ciclo de la prueba**	**Resultados esperados**	**Resultados obtenidos**
1.2.1	Llenado de formularios	Simular 30 usuarios llenando formularios simultáneamente con datos del CSV	Formularios completados sin errores, validación RUT correcta	
1.2.2	Guardado en localStorage	Verificar que los 30 registros se guarden correctamente en localStorage sin pérdida de datos	30 pacientes almacenados, estructura JSON válida, sin corrupción	
1.2.3	Validaciones JavaScript	Comprobar que todas las validaciones del lado del cliente funcionen bajo carga	Validaciones ejecutadas, errores mostrados apropiadamente, sin bloqueos UI	

JUEGO DE DATOS (caso back end)

Clave	**Descripción**
CSV_FILE	data_pacientes.csv
VARIABLE_RUT	${rut} - RUT generado con dígito verificador válido
VARIABLE_NOMBRES	${nombres} - Nombre(s) del paciente desde CSV
VARIABLE_APELLIDOS	${apellidos} - Apellido(s) del paciente desde CSV
VARIABLE_DIRECCIÓN	${direccion} - Dirección completa
VARIABLE_CIUDAD	${ciudad} - Ciudad de residencia
VARIABLE_TELEFONO	${telefono} - Formato: +569xxxxxxxx
VARIABLE_EMAIL	${email} - Email válido del paciente
VARIABLE_FECHA_NAC	${fechaNacimiento} - Formato: YYYY-MM-DD
VARIABLE_ESTADO_CIVIL	${estadoCivil} - Valores: soltero, casado, divorciado, viudo, union
VARIABLE_COMENTARIOS	${comentarios} - Observaciones medicas
WEBDRIVER_SCRIPT	llenarFormulario_webdriver.js
LOCALSTORAGE_KEY	‘pacientes’
VALIDATION_SCRIPT	Función validarRUT () integrada en la aplicación

Caso 1.3: Búsquedas Concurrentes

Descripción: Múltiples usuarios realizando búsquedas de pacientes

Configuración JMeter: Thread Group: 40 usuarios; Ramp-up: 8 segundos; Operaciones de búsqueda variadas

Resultado Esperado: Resultados de búsqueda consistentes, sin bloqueos de interfaz, memoria del navegador estable

CICLOS (caso back end)

Ciclo	Ejecución de la prueba	Objetivo del ciclo de la prueba	Resultados esperados	Resultados obtenidos
1.3.1	Búsqueda básica	40 usuarios realizando búsquedas por apellido simultáneamente	Resultados consistentes, búsqueda <500ms, sin bloqueos de UI	
1.3.2	Filtrado de datos	Verificar que el filtrado de pacientes en localStorage funcione correctamente bajo concurrencia	Datos filtrados correctamente, sin resultados duplicados o faltantes	
1.3.3	Renderizado de resultados	Comprobar que los resultados se muestren correctamente en el DOM para todos los usuarios	Elementos .patient-item creados correctamente, información completa	

JUEGO DE DATOS (caso back end)

Clave	Descripción
SEARCH_TERMS	Array de apellidos: [“García”, “Lopez”, “Gonzalez”, “Martinez”, “Perez”, “Rodriguez”, “Fernandez”, “Sanchez”]
VARIABLE_APELLIDO	${apellido_busqueda} - Apellido aleatorio del array
INPUT_SELECTOR	#searchInput
BUTTON_SELECTOR	#btnBuscar
RESULTS_CONTAINER	#searchResults
WEBDRIVER_SCRIPT	buscarPacientes_webdriver.js
THINK_TIME	Random Timer: 1000-3000 ms entre operaciones
VALIDATION_METHOD	Verificación de elemento .patient-item en DOM
EXPECTED_RESPONSE	JSON con array de pacientes coincidentes
LOCALSTORAGE_READ	localStorage.getItem(‘pacientes’)
FILTER_FUNCTION	paciente.apellidos.toLowerCase().includes(término)

PRUEBA NO FUNCIONAL 2: PRUEBA DE RENDIMIENTO

Consideraciones: Esta prueba evalúa el comportamiento del sistema bajo condiciones normales de carga operativa, simulando múltiples usuarios concurrentes que acceden al formulario de registro de pacientes. Se busca verificar que el sistema mantenga tiempos de respuesta aceptable y no presente errores cuando varios usuarios interactúan simultáneamente con la aplicación.

Función probada (back end)	Operaciones CRUD en localStorage
Objeto probado (front end)	Carga inicial, validación RUT, operaciones localStorage
Responsable de la prueba	Oliver Rubio Rauld

Casos de Prueba de Rendimiento

Caso 2.1: Tiempo de Carga Inicial

Descripción: Medir tiempo de carga completa de la aplicación

Configuración JMeter: Thread Group: 1 usuario; Múltiples iteraciones; Medición de todos los recursos (HTML, CSS, JS)

Resultado Esperado: Carga inicial < 3 segundos; DOM completamente cargado < 2 segundos; Scripts ejecutándose < 0.5 segundos.

CICLOS (caso back end)

Ciclo	**Ejecución de la prueba**	**Objetivo del ciclo de la prueba**	**Resultados esperados**	**Resultados obtenidos**
2.1.1	Primera carga (sin cache)	Medir tiempo de carga completa de la aplicación desde cero	Carga inicial <3s, DOM; cargado <2s, JavaScript; ejecutado <0.5s	
2.1.2	Parsing y	Analizar tiempo de	DOM Content	

	renderizado	parsing HTML/CSS y construcción del DOM	Loaded <2s, First Contentful Paint <1.5s	
2.1.3	Ejecución de scripts	Medir tiempo de inicializacion del JavaScript (event listeners, validaciones)	Scripts ejecutados <0.5s, formulario interactivo y funcional	

JUEGO DE DATOS (caso back end)

Clave	**Descripción**
URL_TARGET	http://localhost:8080/formulario.html
SAMPLER_TYPE	HTTP Request + Response Assertion
RETRIEVE_ALL_RESOURCES	Habilitado para HTML, CSS inline, JavaScript inline
RESOURCE_TIMEOUT	10000 ms por recurso
ITERATION_COUNT	20 ejecuciones consecutivas
CACHE_MANAGER	Deshabilitado para simular primera visita
METRICS_COLLECTED	-DOM Content Loaded - Page Load Complete - First Contentful Paint - Time to interactive
PERFORMANCE_TIMER	BeanShell/JSR223 con window.performance.timing
HTML_SIZE	15-20 KB (estimado)
CSS_SIZE	5 KB (inline en HTML)
JS_SIZE	10-15 KB (inline en HTML)

Caso 2.2: Rendimiento de Validación RUT

Descripción: Evaluar tiempo de procesamiento de validación de RUT

Configuración JMeter: Simulación de entrada de datos; Múltiples RUT válidos e inválidos; Medición de tiempo de validación

Resultado Esperado: Validación instantánea < 100ms; Sin bloqueo de UI durante validación; Feedback visual inmediato.

CICLOS (caso back end)

Campo	**Descripción**
Ejecución de la prueba	Automatizada con WebDriver Sampler
Objetivo del ciclo	Evaluar el tiempo de procesamiento de la función de validación de RUT chileno
Configuración JMeter	WebDriver Sampler: Chrome Iteraciones: 50 RUTs variados: válidos e inválidos Medición de tiempo de ejecución JavaScript
Resultados esperados	Validación instantánea <100ms - Sin bloqueo de UI durante validación - Feedback visual inmediato - Clase .error aplicada correctamente
Resultados obtenidos	

JUEGO DE DATOS (caso back end)

Clave	**Descripción**
RUTS_VALIDOS	[“12345678-9”, “87654321-6”, “11111111-1”, “22222222-2”, “33333333-3”]
RUTS_INVALIDOS	[“12345678-0”, 11111111-2”, “99999999-9”, “12345678-X”, “00000000-0”]
INPUT_SELECTOR	#rut
ERROR_SELECTOR	#rut-error
ERROR_CLASS	.error (aplicada al input)

VALIDATION_FUNCTION	validarRUT (rut) - JavaScript nativo
WEBDRIVER_SCRIPT	validacionRUT_webdriver.js
ITERATION_COUNT	50 (25 válidos + 25 invalidos)
METRICS_TIMER	Performance.now() antes y después de validación
REGEX_FORMAT	/^[0-9]+-[0-9kK]{1}$/
ALGORITHM	Módulo 11 con multiplicadores 2-7
FEEDBACK_VISUAL	-Input con borde rojo si inválido - Mensaje error visible - Clase .show-error en #rut-error
EXPECTED_TIME	<100ms por validación

Caso 2.3: Operaciones LocalStorage

Descripción: Evaluar rendimiento de operaciones CRUD en localStorage

Configuración JMeter: Simulación de guardar múltiples pacientes; Búsquedas en dataset grande; Operaciones de actualización.

Resultado Esperado: Guardar/cargar < 50ms; Búsquedas < 200ms; Sin degradación con dataset grande.

CICLOS (caso back end)

Ciclo	Ejecución de la prueba	Objetivo del ciclo de la prueba	Resultados esperados	Resultados obtenidos
2.3.1	Operación WRITE (guardar)	Medir tiempo de localStorage.setItem() con datasets de 0, 50, 100 y 500 pacientes	Guardar <50ms independiente del dataset previo, sin errores.	

2.3.2	Operación READ (leer)	Medir tiempo de localStorage.getItem() y JSON.parse() con diferentes volúmenes.	Leer y parsear <50ms para 500 pacientes, estructura válida	
2.3.3	Operación SEARCH (buscar)	Medir tiempo de búsqueda/filtrado sobre arrays de diferentes tamaños	Búsqueda <200ms incluso con 500 registros, sin degradación lineal	

JUEGO DE DATOS (caso back end)

Clave	**Descripción**
DATASET_INICIAL	Tres configuraciones: 0 pacientes, 50 pacientes, 100 pacientes, 500 pacientes
GENERATOR_SCRIPT	script_datos_prueba VERSION 2.js
LOAD_FUNCTION	generadorPrueba.cargarEnLocalStorage(cantidad)
LOCALSTORAGE_KEY	‘pacientes’
OPERATION_SAVE	localStorage.setItem(‘pacientes’, JSON.stringify(data))
OPERATION_READ	JSON.parse(localStorage.getItem(‘pacientes’)
OPERATION_UPDATE	Modificar objeto + setItem completo
OPERATION_SEARCH	Array.filter() sobre datos cargados
SAMPLER_TYPE	JSR223 Sampler + WebDriver Sampler
METRICS_COLLECTED	-Tiempo setItem() - Tiempo getItem() - Tiempo JSON.parse() - Tiempo búsqueda/filtrado
DATA_STRUCTURE	JSON Object: {“rut”:{paciente_data}}
PATIENT_SIZE	400-500 bytes por paciente
MAX_DATASET	500 pacientes (250KB de datos)
PERFORMANCE_API	window.performance.mark() /measure()

PRUEBA NO FUNCIONAL 3: PRUEBA DE ESTRÉS

Consideraciones: Esta prueba determina el punto de ruptura del sistema sometiendo a cargas extremas para identificar límites de capacidad y comportamiento bajo condiciones críticas. Se busca encontrar el umbral donde el sistema comienza a fallar o degradarse significativamente.

Casos de Prueba de Estrés

Caso 3.1: Sobrecarga de Usuarios

Descripción: Incrementar usuarios hasta encontrar el punto de falla

Configuración JMeter: Thread Group: Incremental (50, 100, 200, 500 usuarios); Ramp-up: 30 segundos;

Duración: 120 segundos

Resultado Esperado: Identificar punto de saturación; Comportamiento graceful degradation; Sin corrupción de datos

CICLOS (back end)

Ciclo	Ejecución de la prueba	Objetivo del ciclo de la prueba	Resultados esperados	Resultados obtenidos
3.1.1	Fase 1-50	Establecer línea base de rendimiento con carga normal	Sistema estable, tiempos normales, 0% errores	
3.1.2	Fase 2-100	Incrementar a carga moderada-alta y observar comportamiento	Posible aumento de tiempos, sistema funcional, <2% errores	
3.1.3	Fase 3-200	Alcanzar carga alta y detectar primeros signos de degradación	Degradación evidente tiempos >5s, errores <5%	
3.1.4	Fase 4-500	Identificar punto de ruptura o saturación del sistema	Punto de ruptura identificado, errores >5%, posible falla	

JUEGO DE DATOS (caso back end)

Clave	**Descripción**
THREAD_GROUP_TIPO	Stepping Thread Group
FASE_1	50 usuarios - Ramp-up: 10s - Hold: 30s
FASE_2	100 usuarios - Ramp-up: 10s - Hold: 30s
FASE_3	200 usuarios - Ramp-up: 10s - Hold: 30s
FASE_4	500 usuarios - Ramp-up: 10s - Hold: 30s
DURACION_TOTAL	120 segundos
SAMPLERS_MIX	50% carga página, 30% envío formulario, 20% búsqueda
MONITORS_ACTIVOS	- PerfMon Metrics Collector - Response Time Graph - Active Threads Over Time
CPU_THRESHOLD	Alerta si >90%
MEMORY_THRESHOLD	Alerta si >85%
ERROR_THRESHOLD	Alerta si >5%
THROUGHPUT_MIN	> 20 req/s en todas las fases
RECOVERY_TEST	Verificar sistema tras finalizar estrés
LOCALSTORAGE_INTEGRITY	Validar datos no corrompidos post-prueba

Caso 3.2: Memoria y Recursos del Navegador

Descripción: Saturar localStorage y memoria del navegador

Configuración JMeter: Inserción masiva de registros (1000+ pacientes); Operaciones continuas por

períodos extendidos; Monitoreo de consumo de memoria

Resultado Esperado: Manejo adecuado de límites de localStorage; Sin memory leaks; Funcionalidad básica mantenida.

CICLOS (caso back end)

Ciclo	**Ejecución de la prueba**	**Objetivo del ciclo de la prueba**	**Resultados esperados**	**Resultados obtenidos**
3.2.1	Inserción masiva inicial	Cargar 1000+ pacientes en localStorage progresivamente (100 por iteracion)	Inserción exitosa hasta alcanzar límite, QuotaExceededError apropiado	
3.2.2	Monitoreo de memoria JS Heap	Monitorear window.performance.memory durante 300 segundos de operación continua	Memoria usada <límite heap, sin memory leaks (crecimiento constante)	
3.2.3	Recuperación y funcionalidad básica	Limpiar localStorage y verificar que la aplicación siga funcional	Sistema recuperado, funcionalidad básica operativa, formulario usable	

JUEGO DE DATOS (caso back end)

Clave	**Descripción**
GENERATOR_FUNCTION	generadorPrueba.generarDatosEstres(1000)
TARGET_RECORDS	1000+ pacientes (incremento de 100 en 100)
WEBDRIVER_SCRIPT	saturarLocalStorage_webdriver.js
LOCALSTORAGE_LIMIT	5-10 MB (varía por navegador)
INJECTION_RATE	10 pacientes por segundo
DURATION	300 segundos de operación continua
MEMORY_MONITOR	window.performance.memory.usedJSHeapSize
METRICS_COLLECTED	- Memoria usada JS Heap - Límite JS Heap - Total JS Heap Size - Tiempo operaciones localStorage
ERROR_QUOTA	QuotaExceededError cuando se alcanza el límite
EXPECTED_SIZE	500KB por 1000 pacientes
GRACEFUL_HANDLING	Verificar mensaje de error apropiado al usuario
RECOVERY_TEST	Limpiar localStorage y verificar funcionalidad básica
LEAK_DETECTION	Monitorear memoria durante 5 minutos post-carga
BROWSER_CONSOLE	Capturar errores y warnings JavaScript

Caso 3.3: Concurrencia Extrema de Operaciones

Descripción: Múltiples operaciones simultáneas en el mismo dataset

Configuración JMeter: 100+ usuarios realizando operaciones CRUD simultáneas; Duración extendida (300 segundos); Variabilidad de operaciones

Resultado Esperado: Consistencia de datos mantenida; No corrupción de localStorage; Sistema recuperable tras la prueba.

CICLOS (caso back end)

Ciclo	Ejecución de la prueba	Objetivo del ciclo de la prueba	Resultados esperados	Resultados obtenidos
3.3.1	Operaciones CRUD mixtas	100+ usuarios ejecutando operaciones CREATE, READ, UPDATE simultáneas durante 300s	Sistema procesa operaciones, posibles conflictos manejados, datos consistentes	
3.3.2	Validación de consistencia	Verificar integridad de datos en localStorage después de operaciones concurrentes	JSON parseable, estructura válida, todos los RUTs válidos, sin duplicados	
3.3.3	Recuperación del sistema	Comprobar que el sistema sea recuperable y funcional post-prueba de estrés	Sistema operativo, localStorage accesible, funcionalidad completa restaurada	

JUEGO DE DATOS (caso back end)

Clave	Descripción
THREAD_COUNT	100+ usuarios concurrentes
DURATION	300 segundos de operación continua
OPERATION_MIX	40% lecturas (búsquedas), 30% escrituras (nuevos pacientes), 20% actualizaciones, 10% eliminaciones simuladas
BASE_DATASET	100 pacientes pre-cargados en localStorage
CRUD_SCRIPTS	- create_webdriver.js - read_webdriver.js - update_webdriver.js - delete_webdriver.js
RANDOM_SELECTOR	Función JavaScript para seleccionar operación aleatoria
THINK_TIME	500-2000 ms entre operaciones
LOCALSTORAGE_KEY	‘pacientes’

CONFLICT_SCENARIOS	Múltiples usuarios modificando mismo RUT simultáneamente
VALIDATION_POST	Verificar integridad JSON.parse() después de prueba
EXPECTED_RECORDS	100+ nuevos - eliminados (variabilidad esperada)
DATA_CONSISTENCY	Validar estructura de cada registro paciente
RECOVERY_VALIDATION	localStorage.getItem('pacientes') parseable sin errores
ERROR_LOGGING	Capturar QuotaExceededError, SyntaxError, TypeError
CORRUPTION_CHECK	Verificar todos los RUTs válidos post-prueba

CONFIGURACIÓN TÉCNICA JMETER

- *Configuración Plan de Pruebas*

  BASE_URL: http://localhost:8080

  NUM_USUARIOS_CARGA: 50

  NUM_USUARIOS_RENDIMIENTO: 20

  NUM_USUARIOS_ESTRES: 100

  DURACION_CARGA: 60

  DURACION_RENDIMIENTO: 90

  DURACION_ESTRES: 300

- *Thread Groups*

  1.1 Carga_Pagina_Principal

  1.2 Envío_Formularios

  1.3 Búsquedas_Concurrentes

  2.1 Tiempo_Carga_Inicial

  2.2 Rendimiento_Validacion_RUT

  2.3 Operaciones_LocalStorage

  3.1 Sobrecarga_Usuarios_Estres

  3.2 Estrés_Memoria_Recursos

  3.3 Concurrencia_Extrema_Operaciones

- *Samplers*

  HTTP Request Sampler

  WebDriver Sampler (Chrome)

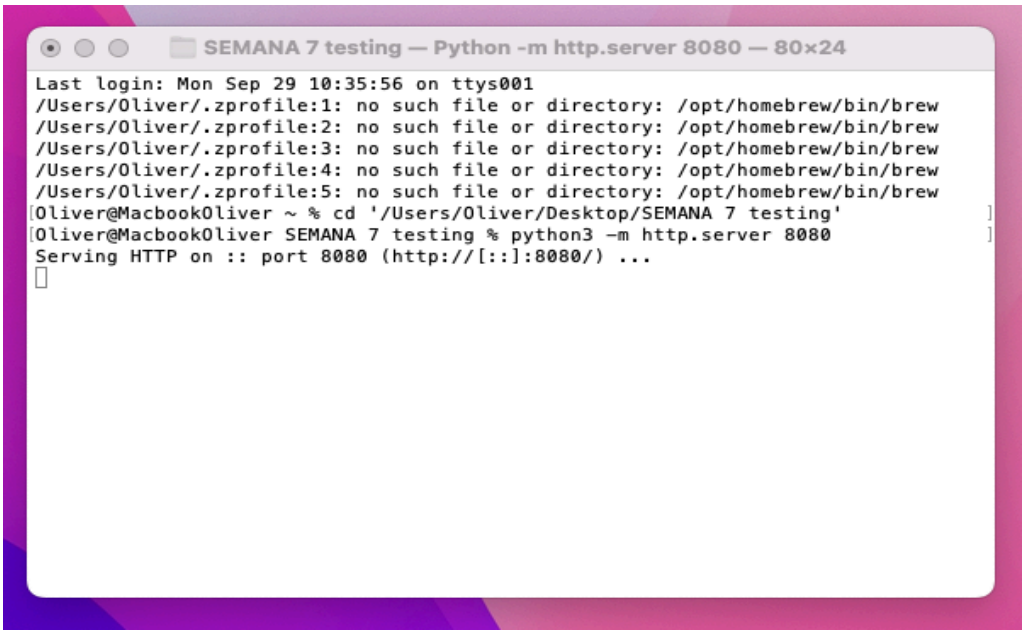
JSR223 Sampler (JavaScript)

- *Receptores Configurados*

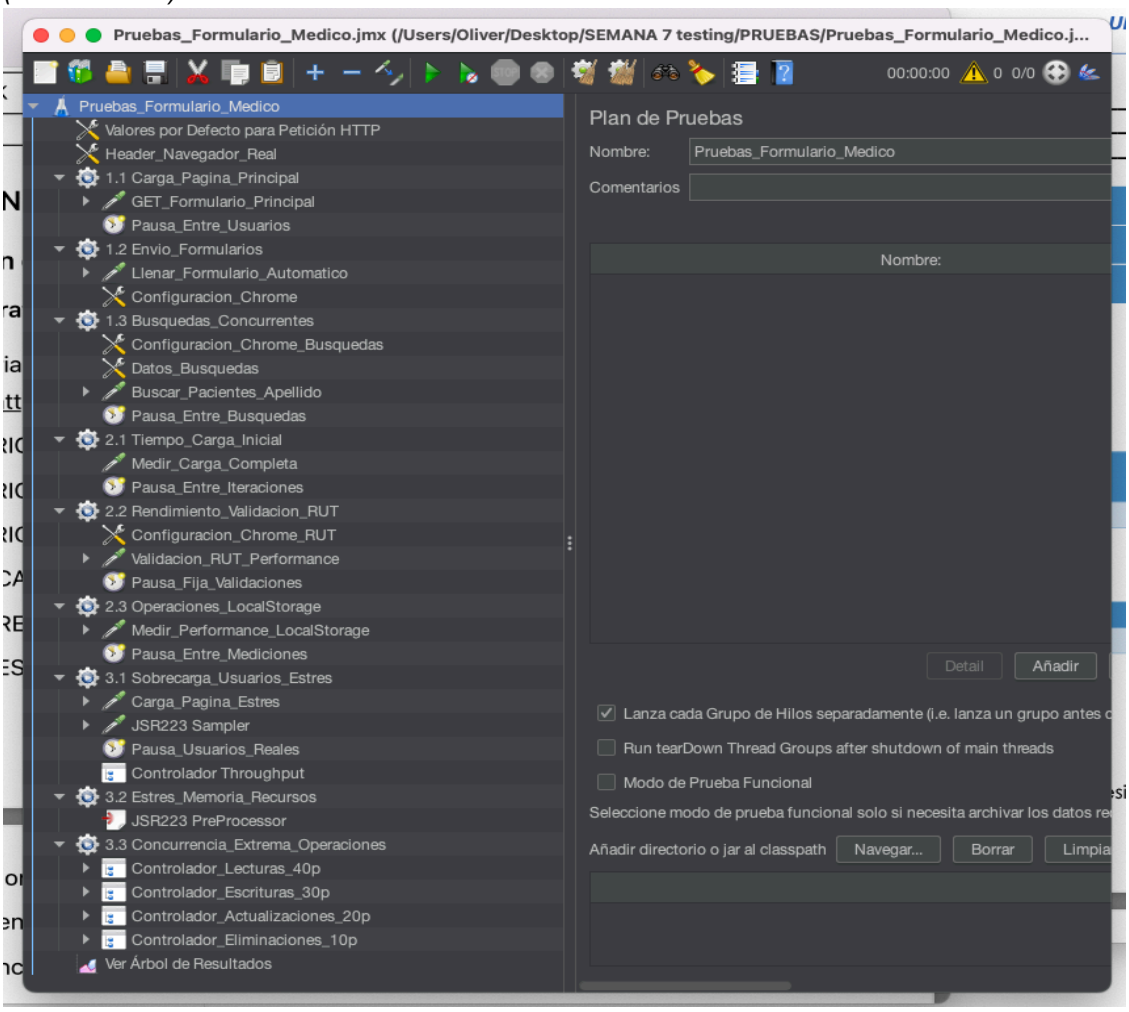
  Ver Árbol de Resultados

CAPTURAS

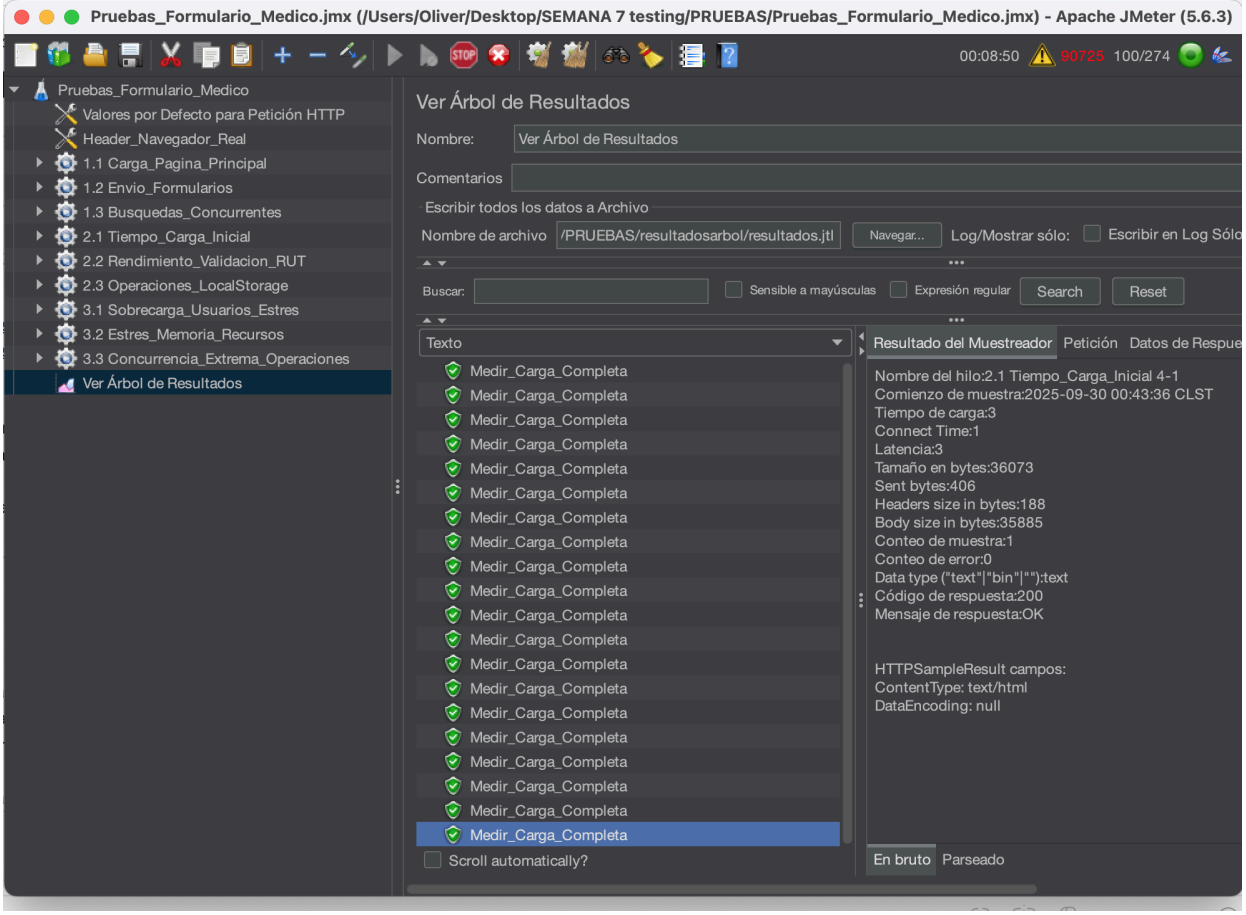
(Localhost)



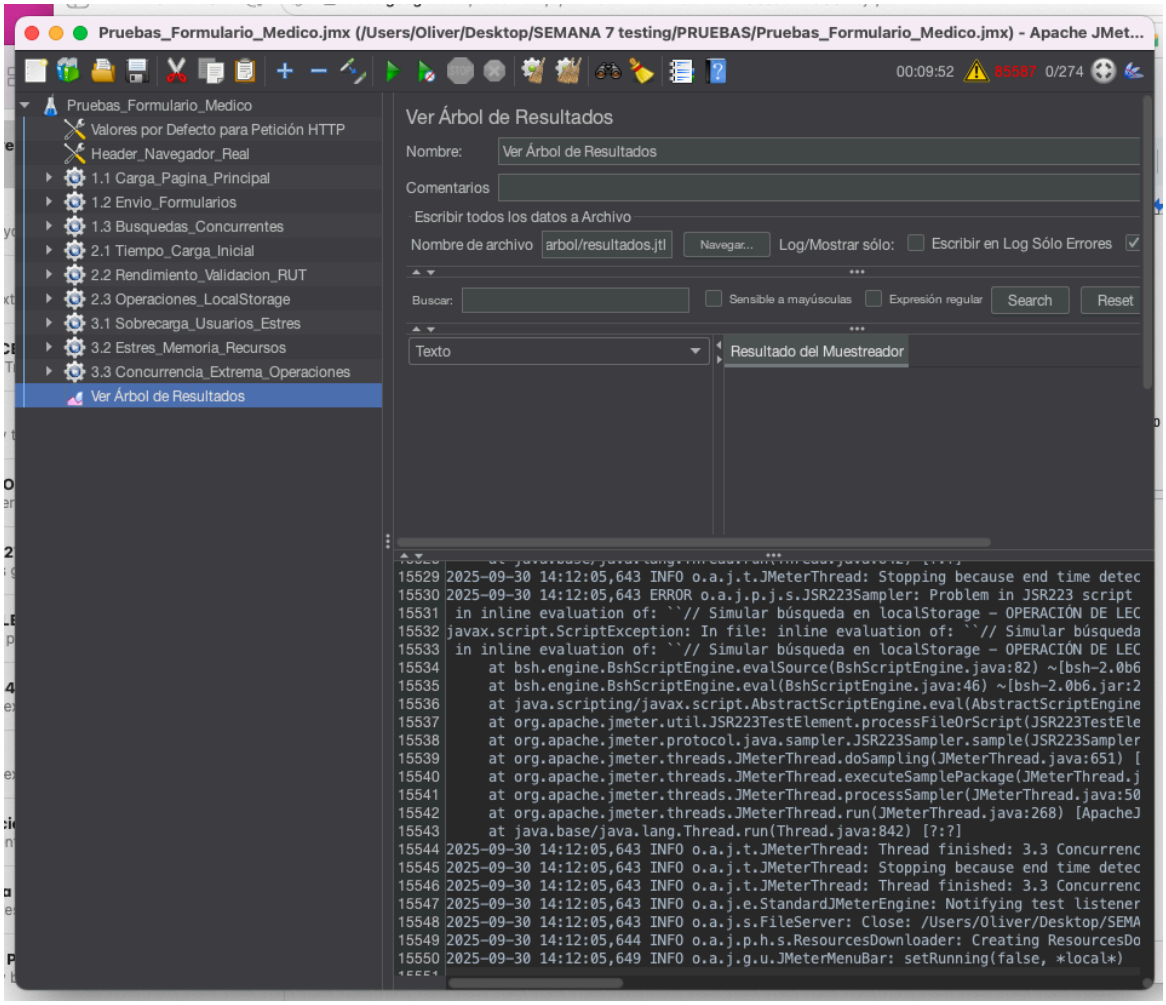
(Panel JMeter)



(Medición de carga página principal)



(Log errores JMeter)



Reflexiones finales

El desarrollo y ejecución de las pruebas no funcionales descritas en el informe representan un esfuerzo integral y bien estructurado para evaluar la calidad de una aplicación web crítica: un formulario de ficha médica que opera en el contexto chileno, utilizando tecnologías web estándar y almacenamiento local. Aunque la ejecución práctica con JMeter enfrentó dificultades técnicas —principalmente incompatibilidades con JDK22 y entornos OSX—, el proceso de planificación, diseño y documentación realizado posee un valor significativo y demuestra una comprensión profunda de los principios de testing de software.

Alineación con Estándares de la Industria y Fundamentos Teóricos

Se aplicaron rigurosamente los principios de las pruebas no funcionales, las cuales, se centran en el "cómo" funciona el sistema y no en el "qué" hace. En lugar de verificar funcionalidades específicas (como el registro de un paciente), el informe se enfoca en atributos de calidad críticos:

Rendimiento: Se diseñaron casos para medir el tiempo de carga inicial (Caso 2.1) y el tiempo de respuesta de operaciones específicas como la validación de RUT (Caso 2.2) y el acceso a localStorage (Caso 2.3).

Carga: Los casos 1.1, 1.2 y 1.3 simularon condiciones de uso normal y concurrente, buscando verificar que el sistema mantuviera un comportamiento estable con múltiples usuarios.

Estrés: Los casos de la Prueba 3 (Sobrecarga de Usuarios, Memoria y Concurrencia Extrema) fueron diseñados específicamente para "determinar el punto de ruptura del sistema". El enfoque incremental (50, 100, 200, 500 usuarios) y la saturación de localStorage son metodologías estándar para evaluar la robustez y confiabilidad del software bajo condiciones extremas.

Enfoque Proactivo y Aprendizaje ante las Dificultades Técnicas

A pesar de que la ejecución automatizada no pudo completarse, el valor del trabajo realizado reside en el proceso de ingeniería aplicado. La incapacidad de JMeter para funcionar con la configuración actual es un recordatorio práctico de un principio fundamental en testing: el entorno de pruebas debe ser compatible y estable. Este obstáculo no invalida el diseño; por el contrario, subraya su importancia.

El diseño meticuloso de los casos de prueba significa que, una vez resueltas las incompatibilidades técnicas (por ejemplo, utilizando una versión compatible del JDK o configurando un entorno virtual), la suite de pruebas está lista para ser ejecutada inmediatamente. El esfuerzo de planificación—que suele ser la parte más compleja—ya está realizado. Además, esta experiencia proporciona un aprendizaje invaluable sobre la gestión de dependencias tecnológicas y la importancia de validar el entorno de testing en las etapas iniciales de un proyecto.

Conclusión

En conclusión, el trabajo descrito en el informe representa un ejercicio de alta calidad en la planificación de pruebas no funcionales. Se logró traducir los requisitos de calidad del sistema (rendimiento, estabilidad bajo carga y resistencia al estrés) en un plan de pruebas concreto, medible y alineado con los estándares de la industria. Las dificultades técnicas encontradas son, en esencia, problemas de implementación y configuración que pueden ser superados, y no reflejan una falla en el diseño conceptual o metodológico.

El enfoque debe permanecer positivo: se ha construido una base sólida y documentada para garantizar la calidad del software. El siguiente paso natural es trasladar este diseño a un entorno de ejecución funcional, donde el valor de este trabajo de planificación se materializará plenamente en resultados tangibles, permitiendo identificar cuellos de botella, optimizar el rendimiento y asegurar una experiencia de usuario fluida y confiable para la aplicación de fichas médicas.

Bibliografía

- Beizer, B. (1995). Software Testing Techniques (2nd ed.). Van Nostrand Reinhold.

- Apache Software Foundation. (2024). JMeter User's Manual. https://jmeter.apache.org/usermanual/index.html

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.